[0x34]

SWAR (SIMD-within-a-register)

(see 2nd page)

```
mov     ecx,eax
and     ecx,0xffff0000
shr     ecx,0x10
and     eax,0x0000ffff
shl     eax,0x10
or      eax,ecx

mov     ecx,eax
and     ecx,0xff00ff00
shr     ecx,0x8
and     eax,0x00ff00ff
shl     eax,0x8
or      eax,ecx

mov     ecx,eax
and     ecx,0xcccccccc
shr     ecx,0x2
and     eax,0x33333333
shl     eax,0x2
or      eax,ecx

mov     ecx,eax
and     ecx,0xf0f0f0f0
shr     ecx,0x4
and     eax,0x0f0f0f0f
shl     eax,0x4
or      eax,ecx

mov     ecx,eax
and     ecx,0xaaaaaaaa
shr     ecx,0x1
and     eax,0x55555555
shl     eax,0x1
or      eax,ecx
```

successive parallel swaps

bit groups of doubling size

[0x34]

```
mov  ecx, eax
and  ecx, 0xffff0000
shr  ecx, 0x10
and  eax, 0x0000ffff
shl  eax, 0x10
or   eax, ecx
```

ecx := eax

ecx → [same (16 bits) 0000 0000 0000 0000 (16 bits)]

eax = [0 ··· 0 (16) 'same']

⇒ ecx = [0 ··· 0 (16) 'same'] = ['same' 0 ··· 0 (16)]

| [0 ··· 0 (16) 'same']

[··· 0 ···]

if eax = f ··· f

is 0s and lower-order bits

(i.e. / 2^{16})

is or [0 'same'; 0 'same'] ⇒ eax = [0 | 'same']

```
mov  ecx, eax
and  ecx, 0xff00ff00
shr  ecx, 0x8
and  ecx, 0x00ff00ff
shl  eax, 0x8
or   eax, ecx
```

ecx := ecx

ecx = [A 0000 0000 B 0000 0000]

eax = [0000 0000 0000 0000 0000 0000 0000 0000 B]

or

ecx = [0000 0000 0000 0000 (B | B) 0000 0000]

ecx := 1100 1100 1100 1100 1100 1100 1100 1100

0000 0000 0000 0000 h₀h₀ i₀i₀ 00 0000 0000

shr 2

0000 0000 0000 0000 H₀H₀ I₀I₀ 00 0000 0000

or ⇒ eax = [··0··· (16 bits) h₀h₀ i₀i₀ | H₀H₀ I₀I₀ 00 0000 0000]

if eax = 0 f ··· f

0x0000ff00

```
mov  ecx, eax
and  ecx, 0xcccccccc
shr  ecx, 0x2
and  eax, 0x33333333
shl  eax, 0x2
or   eax, ecx
```

ecx := 1100 1100 1100 1100 1100 1100 1100 1100

0000 0000 0000 0000 0000 0000 0000 0000

shr 2

0000 0000 0000 0000 0000 0000 0000 0000

0000 0000 0000 0000 0000 0000 0000 0000

0000 0000 0000 0000 0000 0000 0000 0000